

# AI/ML PROJECT SUBMISSION



| PROJECT OVERVIEW                                                                                                                                | STUDENT DETAILS                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Project Number: 3</li><li>• Task Name: Expense Tracker System.</li><li>• Difficulty: Moderate</li></ul> | <ul style="list-style-type: none"><li>• Name: Pranav Gajanan Rane</li><li>• College: Government Polytechnic Gondia</li><li>• Branch: Computer Engineering</li><li>• Domain: AI/ML</li></ul> |



```
from datetime import datetime
```

```
def add_expense(): #add expenses made
```

```
    try:
```

```
        description = input("Describe the expense: ")
```

```
        category = input("Category of the expense: ")
```

```
        date = input("Date of expense (DD-MM-YYYY): ")
```

```
        # validate date format early
```

```
        try:
```

```
            datetime.strptime(date, "%d-%m-%Y")
```

```
        except ValueError:
```

```
            print("Date must be in DD-MM-YYYY format.")
```

```
            return
```

```
        amount = float(input("Enter the amount you spent:
    "))
```

```
        if amount <= 0:
```

```
            print("You cannot make negative or zero
expenses!")
```

```
            return
```

```
        expenses.append({
```

```
            "desc": description,
```

```
            "category": category,
```

```
            "amount": amount,
```

```
            "date": date
```

```
        })
```

```
        print("Expense added successfully!")
```

```
    except ValueError:
```

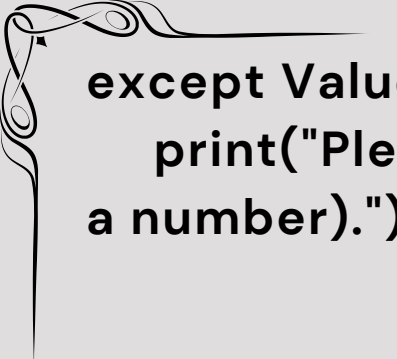


```
print("The book is available to be issued!")
library[issue]['available'] = False
    name = input("Enter your name - ")
    library_id = int(input("Enter your library id - "))
    library[issue]['borrower_name'] = name
    library[issue]['borrower_id'] = library_id
    today = date.today()
    library[issue]['issue_date'] = today
    due_date = today + timedelta(days=7)
        print(f"You need to return the book by -
        {due_date}")
        print(f"You, {library[issue]['borrower_name']}
        have successfully borrowed the book!")
        library[issue]["due_date"] = due_date

    else :
        print("The book is unavailable to be borrowed! ")

    else :
        print("The book with this ISBN isn't available in our
        library!")
    except ValueError :
        print("Pls enter correct value!")

def return_book() : #Returns a book
    try :
        issue = input("Enter the ISBN of book to be returned
        - ")
```



```
except ValueError:
```

```
    print("Please enter correct values (amount must be  
a number).")
```

```
def view_expenses(): # shows all expenses made
```

```
    if not expenses:
```

```
        print("No expenses made!")
```

```
    else:
```

```
        for index, expense in enumerate(expenses, start=1):
```

```
            print(f"{index}) {expense['desc']}")
```

```
            print("Expenses made for this -")
```

```
            print(f"Category - {expense['category']}")
```

```
            print(f"Amount - Rs.{expense['amount']:.2f}")
```

```
            print(f>Date - {expense['date']}")
```

```
            print("=" * 30)
```

```
def category_summary(): # showcases category-wise  
spending
```

```
    if not expenses:
```

```
        print("No expenses made!")
```

```
    return
```

```
c_summary = {}
```

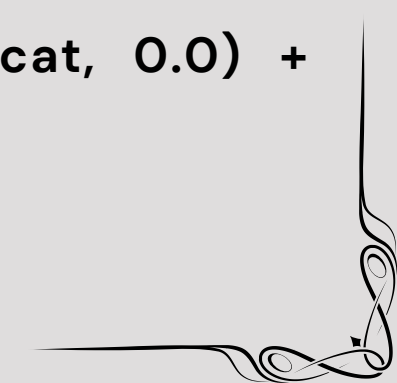
```
for expense in expenses:
```

```
    cat = expense["category"]
```

```
        c_summary[cat] = c_summary.get(cat, 0.0) +  
expense["amount"]
```

```
print("Category-wise summary:")
```

```
for category, total in c_summary.items():
```





```
print(f"{category}: Rs. {total:.2f}")
```

```
def budget_report(budget): # Calculate budget report
```

```
    total_spent = sum(exp["amount"] for exp in expenses)
```

```
    print(f"Total spent: Rs.{total_spent:.2f}")
```

```
    print(f"Budget: Rs.{budget:.2f}")
```

```
    if total_spent > budget:
```

```
        print("WARNING!! - You have exceeded your budget!")
```

```
    else:
```

```
        remaining = budget - total_spent
```

```
        used_pct = (total_spent / budget) * 100 if budget > 0 else 0
```

```
        print(f"You are within budget. Remaining: Rs. {remaining:.2f}")
```

```
        if used_pct >= 100:
```

```
            print("You have used 100% of your budget(overbudget).")
```

```
        elif used_pct >= 80:
```

```
            print(f"WARNING: You have used {used_pct:.2f}% of your budget (>=80%).")
```

```
        else:
```

```
            print(f"Budget used: {used_pct:.2f}%.")
```

```
def top_category(): # most expensive category
```

```
    if not expenses:
```

```
        print("No expenses made yet.")
```

```
    return
```



```
summary = {}
for exp in expenses:
    category = exp["category"]
    summary[category] = summary.get(category, 0.0) +
exp["amount"]
```

```
top = max(summary, key=summary.get)
print(f"Top Category: {top} (Rs.{summary[top]:.2f})")
```

```
def most_expensive(): #calculate the most expensive
purchase made
```

```
    if not expenses:
```

```
        print("No expenses recorded yet.")
```

```
    else:
```

```
        max_exp = max(expenses, key=lambda x:
x["amount"])
```

```
        print(f"Most expensive expense: {max_exp['desc']} |
Rs.{max_exp['amount':.2f} | {max_exp['date']}")
```

```
def average_daily_spending(): #bonus task
```

```
    if not expenses:
```

```
        print("No expenses made!.")
```

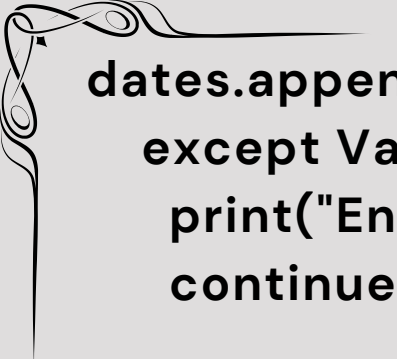
```
    return
```

```
dates = []
```

```
for exp in expenses:
```

```
    try:
```

```
        d = datetime.strptime(exp["date"], "%d-%m-
%Y").date()
```



```
dates.append(d)
```

```
except ValueError:
```

```
    print("Enter correct date!")
```

```
    continue
```

```
if not dates:
```

```
    print("No valid dates found in expenses.")
```

```
    return
```

```
min_date = min(dates)
```

```
max_date = max(dates)
```

```
days_span = (max_date - min_date).days + 1
```

```
    total_spent = sum(exp["amount"] for exp in  
expenses)
```

```
    avg_per_day = total_spent / days_span if days_span  
> 0 else total_spent
```

```
    print(f"Expense date range: {min_date.strftime('%d-  
%m-%Y')} to {max_date.strftime('%d-%m-%Y')}  
( {days_span} day(s) )")
```

```
    print(f"Total spent: Rs.{total_spent:.2f}")
```

```
    print(f"Average daily spending over this range: Rs.  
{avg_per_day:.2f}")
```

```
def show_menu():
```

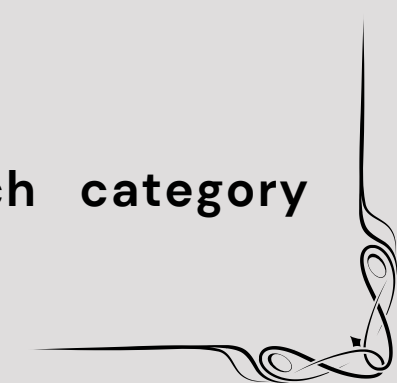
```
    print("\nWhat you can perform - ")
```

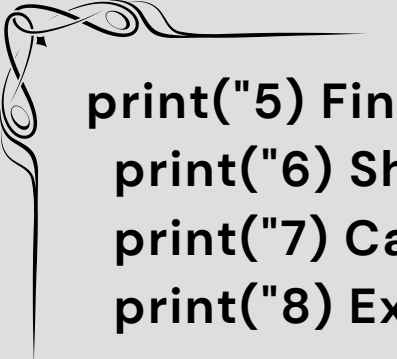
```
    print("1) Add Expense")
```

```
    print("2) View all expenses")
```

```
    print("3) See total expense for each category  
(summary)")
```

```
    print("4) Calculate your Budget Report")
```





```
print("5) Find the most expensive single expense")
print("6) Show top spending category")
print("7) Calculate average daily spending")
print("8) Exit")
```

**'''Actual System Below '''**

```
expenses = [] # globally stores
```

```
def main():
```

```
    try:
```

```
        budget = float(input("Set your monthly budget: "))
```

```
        if budget <= 0:
```

```
            print("Budget must be positive!")
```

```
            return
```

```
    while True:
```

```
        print("\n_____Welcome to Budget
Tracker_____")
```

```
        show_menu()
```

```
        choice = input("Enter what you want to do (1-8) - ")
```

```
        if choice == "1":
```

```
            add_expense()
```

```
        elif choice == "2":
```

```
            view_expenses()
```

```
        elif choice == "3":
```

```
            category_summary()
```

```
        elif choice == "4":
```



```
budget_report(budget)
    elif choice == "5":
        most_expensive()
    elif choice == "6":
        top_category()
    elif choice == "7":
        average_daily_spending()
    elif choice == "8":
        print("Exiting...")
        break
    else:
        print("Enter valid choice only (1-8)!")
except ValueError:
    print("Please enter correct values for budget.")
```

```
if __name__ == "__main__":
    main()
```

*Note - This code might break due to incorrect indentation while pdf making so pls use the code mentioned in .py file attached*



## TEST OUTPUTS -

```
_____Welcome to Budget Tracker_____
```

```
What you can perform -
```

- 1) Add Expense
- 2) View all expenses
- 3) See total expense for each category (summary)
- 4) Calculate your Budget Report
- 5) Find the most expensive single expense
- 6) Show top spending category
- 7) Calculate average daily spending
- 8) Exit

```
Enter what you want to do (1-8) - 1
```

```
Describe the expense: For health
```

```
Category of the expense: Health
```

```
Date of expense (DD-MM-YYYY): 23-2-2026
```

```
Enter the amount you spent: 400
```

```
Expense added successfully!
```

```
Enter what you want to do (1-8) - 2
```

- 1) For health

```
Expenses made for this -
```

```
Category - Health
```

```
Amount - Rs.400.00
```

```
Date - 23-2-2026
```

```
=====
```

- 2) Food

```
Expenses made for this -
```

```
Category - Food
```

```
Amount - Rs.100.00
```

```
Date - 24-2-2026
```

```
=====
```

- 3) Health

```
Expenses made for this -
```

```
Category - haelth
```

```
Amount - Rs.7341.00
```

```
Date - 31-3-2026
```

```
=====
```

\_\_\_\_\_Welcome to Budget Tracker\_\_\_\_\_

What you can perform -

- 1) Add Expense
- 2) View all expenses
- 3) See total expense for each category (summary)
- 4) Calculate your Budget Report
- 5) Find the most expensive single expense
- 6) Show top spending category
- 7) Calculate average daily spending
- 8) Exit

Enter what you want to do (1-8) - 3

Category-wise summary:

Health: Rs. 400.00

Food: Rs. 100.00

haelth: Rs. 7341.00

\_\_\_\_\_Welcome to Budget Tracker\_\_\_\_\_

What you can perform -

- 1) Add Expense
- 2) View all expenses
- 3) See total expense for each category (summary)
- 4) Calculate your Budget Report
- 5) Find the most expensive single expense
- 6) Show top spending category
- 7) Calculate average daily spending
- 8) Exit

Enter what you want to do (1-8) - 4

Total spent: Rs.7841.00

Budget: Rs.10000.00

You are within budget. Remaining: Rs.2159.00

Budget used: 78.41%.

Enter what you want to do (1-8) - 4

Total spent: Rs.9541.00

Budget: Rs.10000.00

You are within budget. Remaining: Rs.459.00

WARNING: You have used 95.41% of your budget (>=80%).

Enter what you want to do (1-8) - 5  
Most expensive expense: Health | Rs.7341.00 | 31-3-2026

Expense date range: 23-02-2026 to 31-03-2026 (37 day(s))  
Total spent: Rs.9541.00  
Average daily spending over this range: Rs.257.86

## **TEST OUTPUTS IN BROKEN CASES -**

```
Date of expense (DD-MM-YYYY): 8  
Date must be in DD-MM-YYYY format.  
  
_____Welcome to Budget Tracker_____
```

**CODE TESTING WAS DONE SUCCESSFULLY!**

***More Details about the project is mentioned in the .md file attached . This pdf was made specifically to attach the code and its outputs.***

PROJECT COMPLETED

